Implementation of Azure Secure Deployment and Monitoring Environment

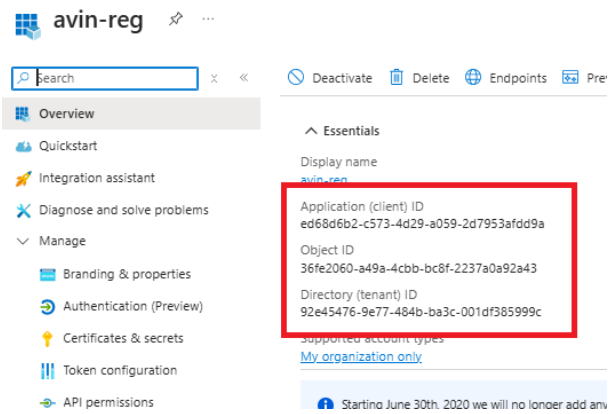
Azure Subscription 1 Region: UK South Tenants: Default Directory.

Authentication of Git Hub to Deploy to Azure Objective To implement a secure method for CI/CD from GitHub to Azure which eliminates the need for the storage of secrets. Methodology OpenID Connect (OIDC) as the Federated Authentication. Allows for identity token issuance by GitHub (Short lived vs. Client Secrets).

Implementation Steps

An App Registration (avindemo) has been created within Microsoft Entra ID.

Step 1 – Creation of App Registration



Shows:

• Application (Client) ID • Directory (Tenant) ID • Display Name Indicates the Azure identity that GitHub will use to authenticate.

Step 2 – Configure Federated Credential

Inside: App Registration → Certificates and Secrets → Federated Credentials

Configured:

• Scenario – GitHub Actions Deploying to Azure Environments

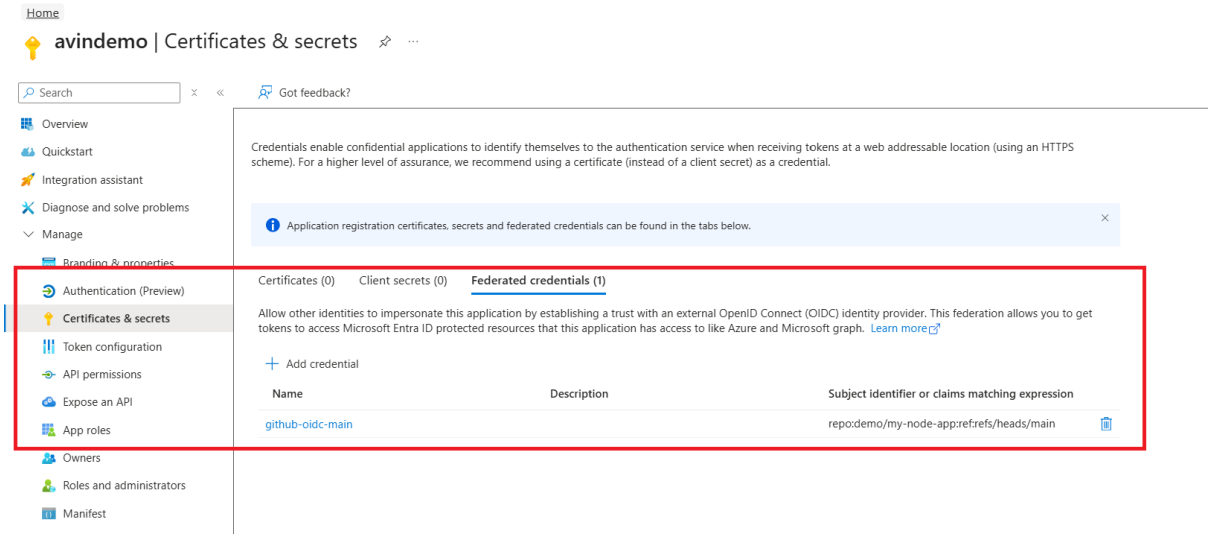
• Organization – demo

• Repository – my-node-app

• Branch – Main

• Subject Identifier

repo:demo/my-node-app:ref:refs/heads/main



This proves:

- No client secret is used
- Trust is established between GitHub and Azure
- Only the main branch can authenticate

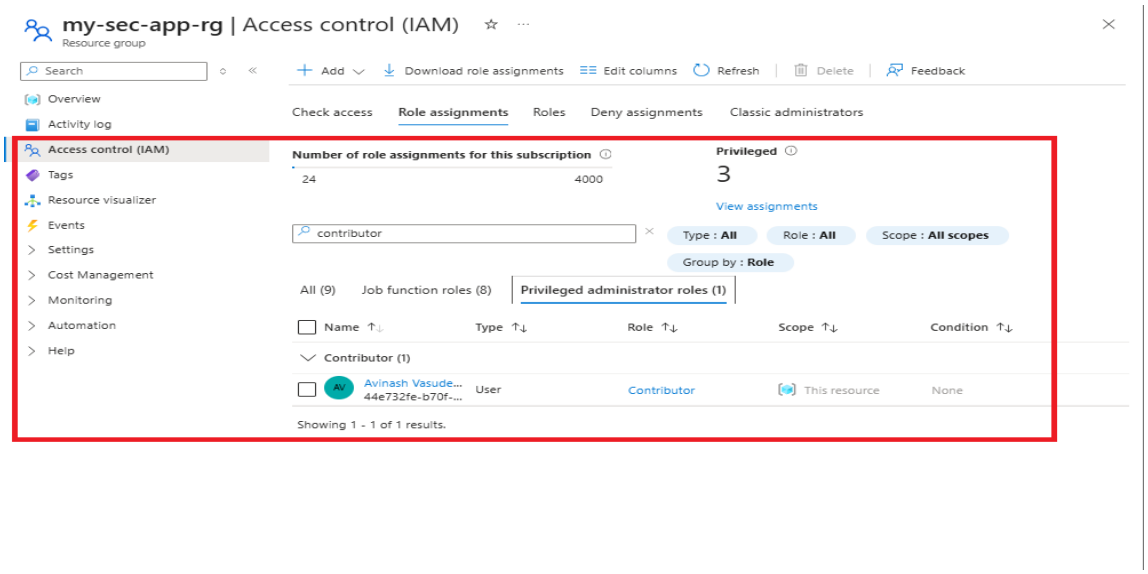
Step 3 – Assign RBAC Role

Resource Group Access Control (IAM) Role Assignments

Assigned:

• Role: Contributor

• Assigned to: App Registration (Service Principal)



This proves:

• The identity has permission to deploy resources

• Access is scoped to the resource group (least privilege)

Step 4 - GitHub Actions Workflow

Workflow configured with:

permissions:

id-token: write

contents: read

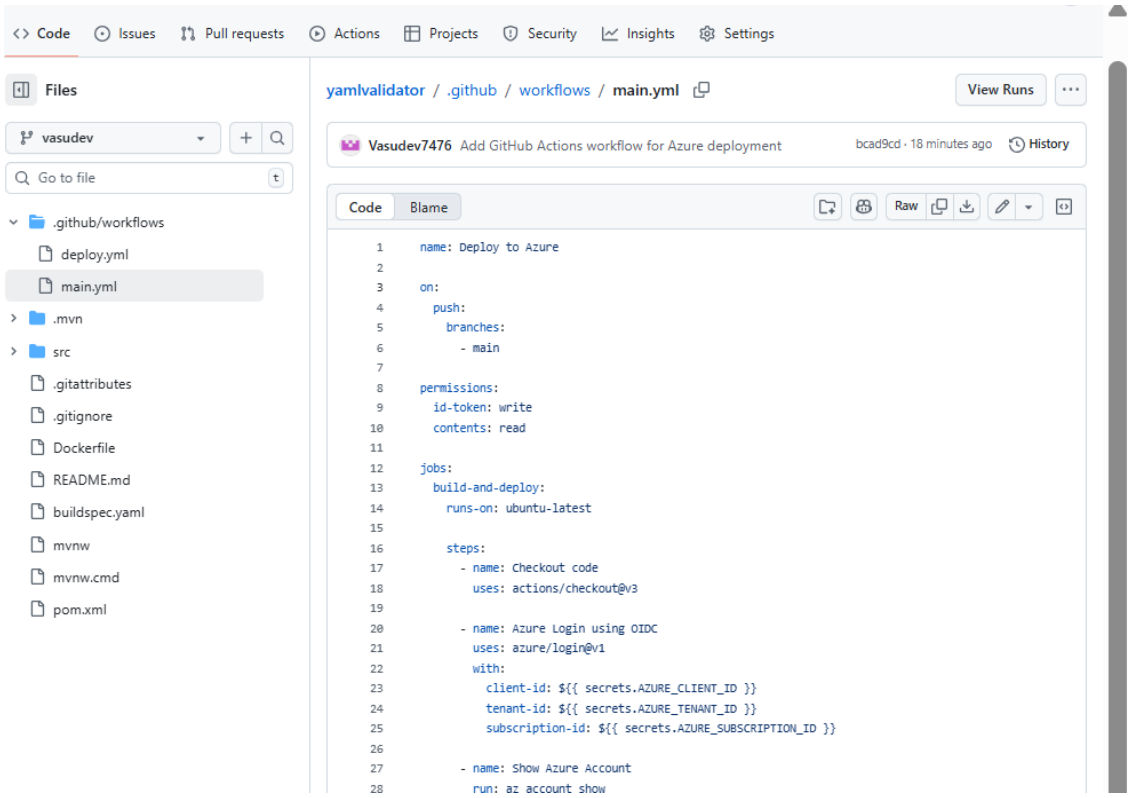
- uses: azure/login@v1

with:

client-id: ${{ secrets.AZURE_CLIENT_ID }}

tenant-id: ${{ secrets.AZURE_TENANT_ID }}

subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}



This

This proves:

• OIDC authentication succeeded

• Azure login was successful

• Deployment pipeline is functional

Security Benefits

• No stored credentials

• Short-lived tokens

• Branch-restricted authentication

• Scoped RBAC access

This follows modern Zero Trust and DevSecOps principles.

Microsoft Sentinel Configuration

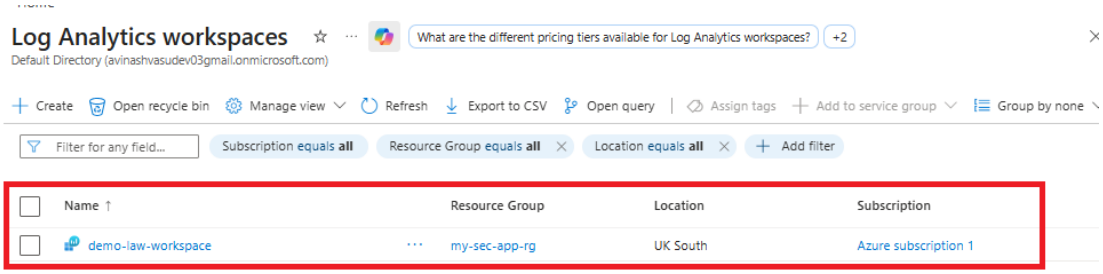
Objective

Ensure visibility and detection across the Azure tenant.

Step 1 - Deploy Log Analytics Workspace

Created:

demo-law-workspace



This is the data ingestion engine for Sentinel.

Step 2 - Enabled Microsoft Sentinel

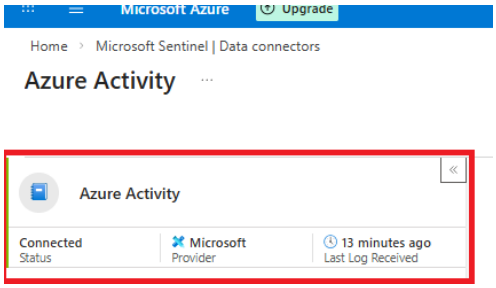
Enabled Sentinel on the workspace.

This activates SIEM capabilities.

Step 3 - Configured Data Connectors

Connected:

- Azure Activity Logs
- Microsoft Defender connectors
- Microsoft Entra ID logs (where applicable)

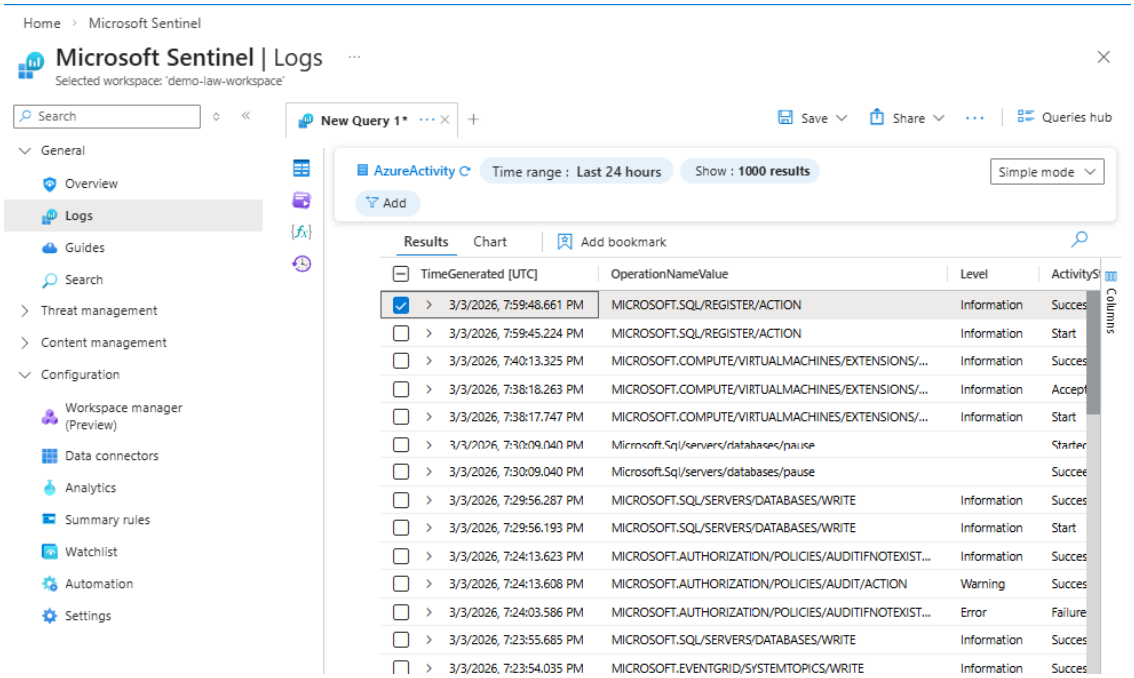


This proves log ingestion is active.

Step 4 - Validate Log Ingestion

Executed query:

AzureActivity



This

This proves:

- Logs are successfully ingesting
- Monitoring is operational

Security Coverage Achieved

• Azure control plane activity monitoring

• Identity monitoring (Entra ID)

• Defender threat signals

• Centralized SIEM visibility

This ensures issues inside the Azure tenant can be detected.

Secure Azure Application Architecture

Objective

Deploy a simple secure web application using:

• App Service

• Azure SQL Database

• Storage Account

• Key Vault

Architecture Overview

Resource Group: my-sec-app-rg

Contains:

• Web App

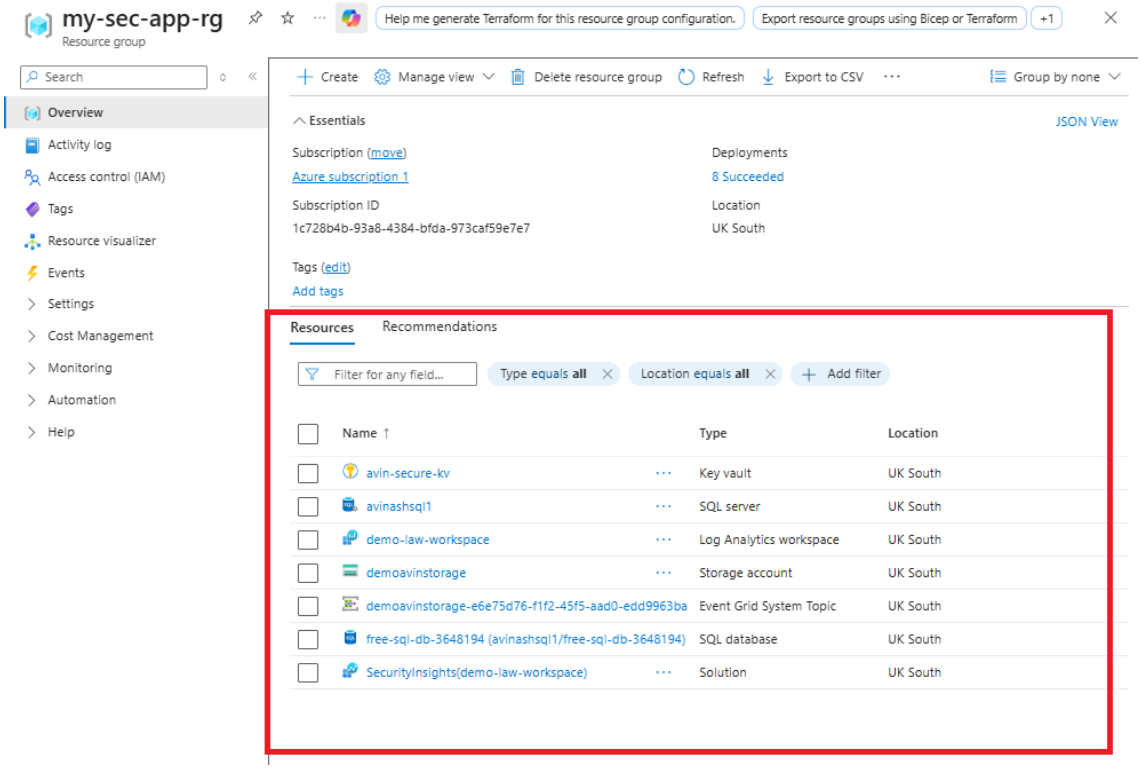
• SQL Server & Database

• Storage Account

• Key Vault

• Log Analytics Workspace

Resource Group OverviewResource Group Overview



This proves all resources are logically grouped.

Step 1 - App Service Deployment

Configured:

• Runtime: Node.js 24 LTS

• OS: Linux

• Region: UK South

• Pricing Tier: Basic B1

Security Controls:

• HTTPS enforced

• Secure hostname enabled

• Managed Identity enabled (recommended)

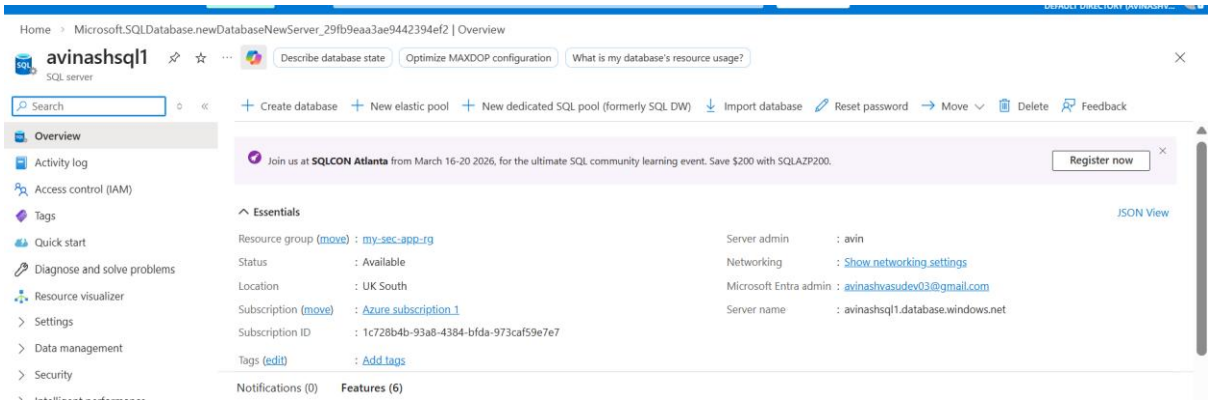
Step 2 – Azure SQL Database

Created:

- SQL Server: avinashsql1

- Database: free-sql-db

SQL Server Overview



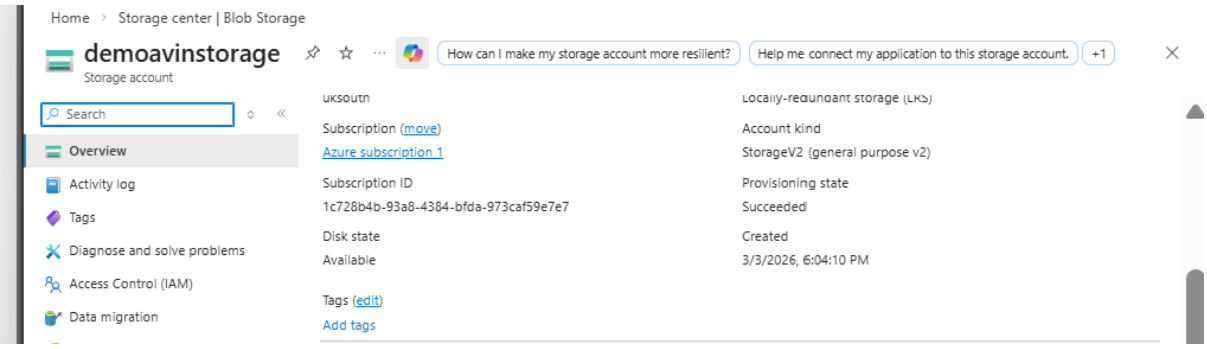
Security Considerations:

- Entra ID Admin configured
- Firewall rules restricted (recommended: disable public access or use private endpoint)

Step 3 - Storage Account

Created:

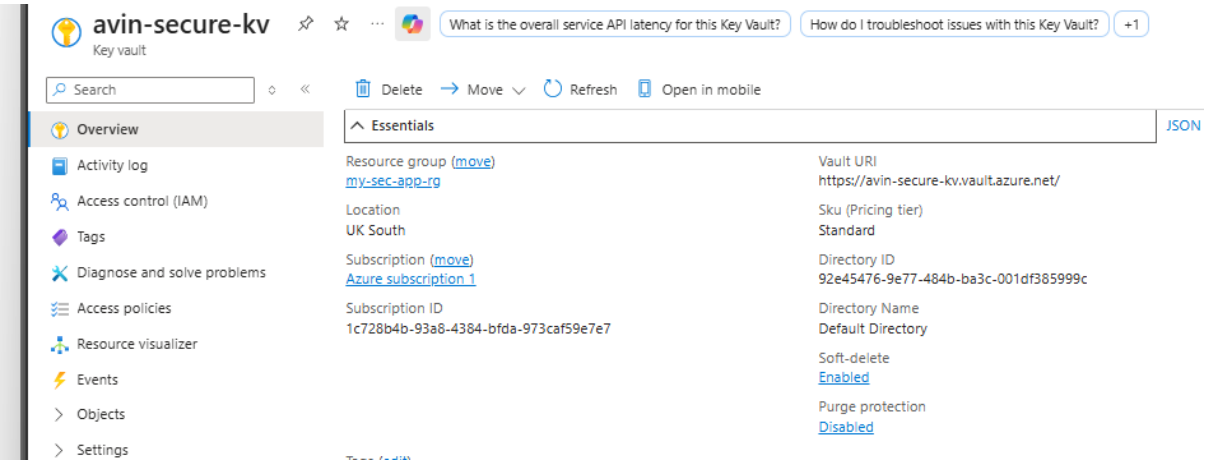
demoavinstorage



Step 4 - Azure Key Vault

Created:

avin-secure-kv



Security Controls:

• Soft delete enabled

• RBAC access control

• Secrets isolated from code

Recommended improvement:

• Enable purge protection for production

Step 5 - Identity-Based Access

App Service should use:

• System Assigned Managed Identity

• Access granted to Key Vault

This removes passwords from configuration.

Overall Security Model

The environment follows:

• Identity-based authentication (OIDC)

• RBAC access control

• Centralized logging

• Secretless deployments

• Encrypted storage